

面向动态智能体任务的按步执行与信息共享

Yaojunzhe Zhai¹ · Chao-Hsien Hsieh²

¹ Monash University Malaysia ² Xi'an International University

摘要

智能体在运行中发现新步骤时，执行位置与输入需求可能同时改变。一次性裁剪既可能沿工具链传播无关信息，也可能遗漏后续计算所需事实。本文演示一个按步执行系统：为当前接收方构造视图，允许受限事实补充或本地表达式计算，并在发送前核验源值、授权与最终请求。界面将任务步骤、实际外发内容和本地计算依赖逐项关联。对 60 页公开财务表格进行五方法、两次重复的冻结验证，共 600 次任务与 1,210 次真实模型调用。本地计算通过 98/120 次，全量数值通过 96/120 次；前者未传输业务数值，但差异区间不足以证明准确率优势。补充分歧视图增加了调用成本，未稳定改善结果。系统据此展示信息共享与任务完成之间可复核的取舍。

关键词：智能体执行；数据视图；按需取数；本地计算；系统演示。

1 问题与系统目标

一份合同规定按延期天数计费。云端已收到条款，但延期天数仍在本地；如果不能申请补充，模型可能把“未传输”误判成“源数据不存在”。随后出现的资料查询只需要编号和版本，不需要合同金额或身份。任务推进改变了信息需求，执行位置和发送内容应随当前步骤重新确定。

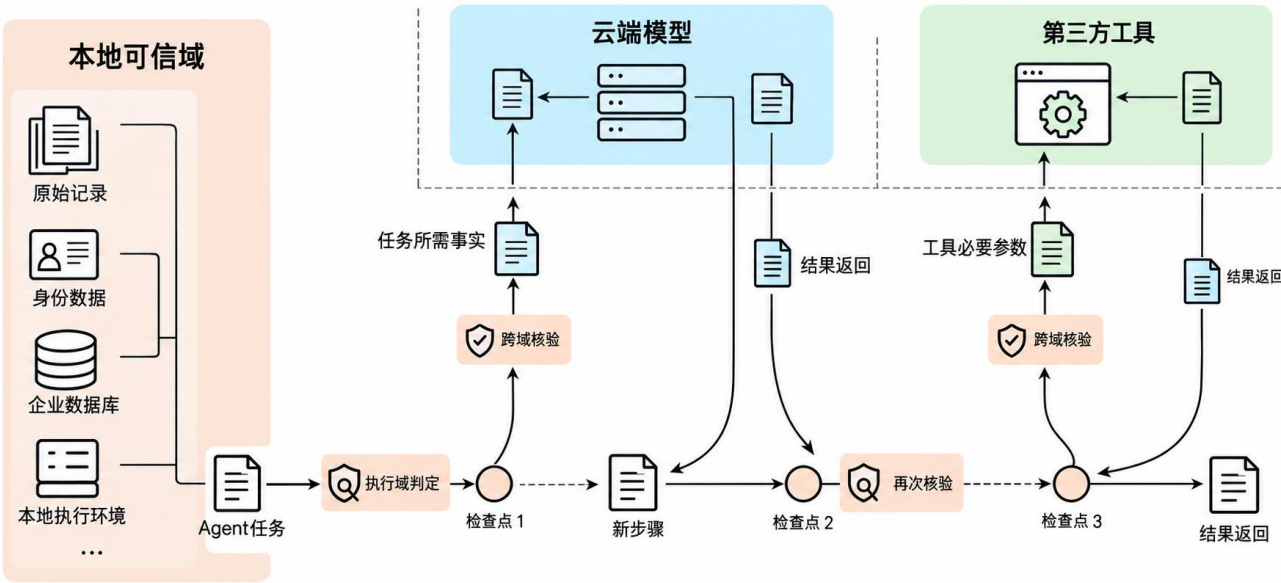


图1 按步执行与出域核验。原始资料在本地保存；模型和工具只通过登记出口交互。任务产生新步骤后重新确定执行位置与输入，结果回到本地状态。

本原型围绕三个可观察的决定展开：哪些操作可由本地能力完成；外部接收方当前需要看到什么；准备好的请求到真正发送时是否仍获准。它既要防止无关上下文沿工具链继续传播，也要允许任务在信息不足时继续执行。

PlanTwin [1] 研究本地规划抽象与受限能力；ToolMinimize [2] 和 MINIM [3] 分别约束工具参数与观察视图。程序辅助推理 [4] 与多方案一致性 [5] 也已有研究。本地网关、程序计算和方案投票本身均不是本文的独创。

本文面向数据管理中的运行时访问控制与计算放置问题，将接收方视图、源值依赖和实际请求纳入同一执行记录。观众可以沿一次任务追溯“为何在此处计算、为何发送这些字段、实际发送是否一致”，而不只看到最终回答。

系统实现事实补充与本地计算两条路径，并加入按候选算式分歧选择有界补充字段的实验机制。贡献包括可核验的执行原型、保持表结构的数值视图，以及带同能力对照和失败记录的演示。我们不将句法最小集合等同于语义最小共享。

2 按步执行机制

2.1 执行位置与接收方视图

本地状态 S 保存原始事实与查询结果。对登记操作 o ，本地规则适用时直接执行；否则为接收方 r 选择字段 K ，并构造视图：

$$V_t = \pi_{K_t}(S_t) \quad (1)$$

t 表示一次外部调用。初始选择器使用登记字段与文本规则，可能漏选。模型可以申请字段 G ；本地核验存在性、授权、新增性及轮数，再更新 K ：

$$K_{t+1} = K_t \cup G_t, G_t \subseteq A(o, r) \quad (2)$$

$A(o, r)$ 是独立于评价标签的授权集合，排除身份、联系方式、账号、内部备注及无关记录。获准不等于必要。合同与学习任务最多补充两轮、分析六轮；越权或超限即失败。

若模型请求资料，控制器另建仅含登记编号与版本的工具视图，将返回正文与出处写入 S 。给模型的授权不会自动延伸至资料服务；原型只支持登记操作。

2.2 将可表达的计算留在本地

数值任务可由云端根据问题与表结构提出有限表达式 p ，参数限于获准字段、常数或前序结果。本地解释器拒绝未知算子、循环引用、非有限结果及无用中间节点，不执行生成脚本。

解释器返回计算值及实际读取的字段集合 $\text{Read}(p)$ 。记录的依赖摘要为：

$$D_t = H(S_t[\text{Read}(p_t)]) \quad (3)$$

实现逐字段保存摘要并在本地组装结果。云端可见问题、标签、年份和单位，不接收业务数值或计算结果。依赖记录不能证明程序理解正确，也不能排除从结构推断信息。

转换器保留原始行列位置、年份和单位，用字段标识替换业务数值。无法确定的合并表头保持原有网格。运行时从原表重建并核对缓存，防止结构与源值不一致。

2.3 按分歧选择补充字段

实验模式规范化三个候选算式。对每对不同候选，依赖不同则取对称差，依赖相同但算子不同则取并集，得到集合 C 。可行 K 须获授权、至多含三个字段，且与每个 C 相交。选择：

$$K^* = \arg \min_K |K| \quad (4)$$

分支搜索保留最小基数解，标识顺序打破平局；无法覆盖则停止。复核接收候选程序与获准字段，不接收本地结果。至多三组句法约束下的最小覆盖，不保证语义充分或答案正确。

2.4 复核最后实际发送的内容

视图确定后，请求仍可能被改写。控制器为最终序列化载荷 B 建立一次性凭据，绑定操作、接收方、授权版本 v 、所用源字段摘要 D 与请求摘要：

$$T_t = (o_t, r_t, v_t, D_t, H(B_t)) \quad (5)$$

发送前任一绑定项改变即拒绝。内部凭据与展示副本分离，避免修改展示元数据影响源值核验。接收进程记录实际字节；适配器另外记录加入固定非思考参数后的供应商载荷，两者由复算脚本对应核验。

算法 1 按步执行与出域判定

Input: state S , operation registry R , policy P
Output: local result and checked execution trace

```
1: while a registered operation  $o$  is pending do
2:   if LocalRule( $o, S$ ) is applicable then
3:      $S \leftarrow \text{Merge}(S, \text{LocalRule}(o, S))$ ; continue
4:   ( $r, K$ )  $\leftarrow \text{InitialView}(o, R, S)$ 
5:   repeat within the registered round limit
6:      $B \leftarrow \text{Serialize}(o, \text{Project}(S, K))$ 
7:      $T \leftarrow \text{Bind}(o, r, P.\text{version}, S[K], B)$ 
8:     RequireCurrentAndUnchanged( $T$ )
9:      $y, \text{receipt} \leftarrow \text{SendOnce}(T, B)$ 
10:    RequireMatchingReceipt(receipt,  $B$ )
11:    if  $y$  requests additional fields  $G$  then
12:      RequireAllowedNewFields( $G, o, r$ )
13:       $K \leftarrow K \cup G$ 
14:    else if  $y$  requests local program  $p$  then
15:       $z, D \leftarrow \text{InterpretChecked}(p, S)$ 
16:       $S \leftarrow \text{Merge}(S, z, D)$ ; break
17:    else if  $y$  requests a reference then
18:       $S \leftarrow \text{Merge}(S, \text{CheckedLookup}(y))$ 
19:       $K \leftarrow \text{RebuildView}(o, r, S)$ 
20:    else  $S \leftarrow \text{Merge}(S, \text{Validate}(y))$ ; break
21:  RequireOperationComplete( $o$ )
22: return AssembleLocally( $S$ ), Trace
```

算法中的操作接口和轮数由登记表限定。模型提出的新步骤保存父步骤和工具调用标识；本地补充与下一次实际发送分别记录。控制器及适配器须可信，性质只覆盖登记出口；它不能阻止网络旁路，也不能撤回已经发送的信息。

2.5 依赖检查的含义与代价

依赖只包含本步使用的源值。修改相关数值会使已准备请求失效，修改未使用字段不影响当前凭据。外发字段与本地计算依赖分开记录：本地读取不等于对外发送；本次未发送也不等于此前从未发送。

有限表达式含 m 个节点时，解释器的顺序执行与依赖遍历需要 $O(m)$ 时间和空间。请求核验随使用字段与载荷长度线性增长。保证范围限于登记出口的访问检查、字节一致性与可重算性，不保证问题理解或信息推断安全。

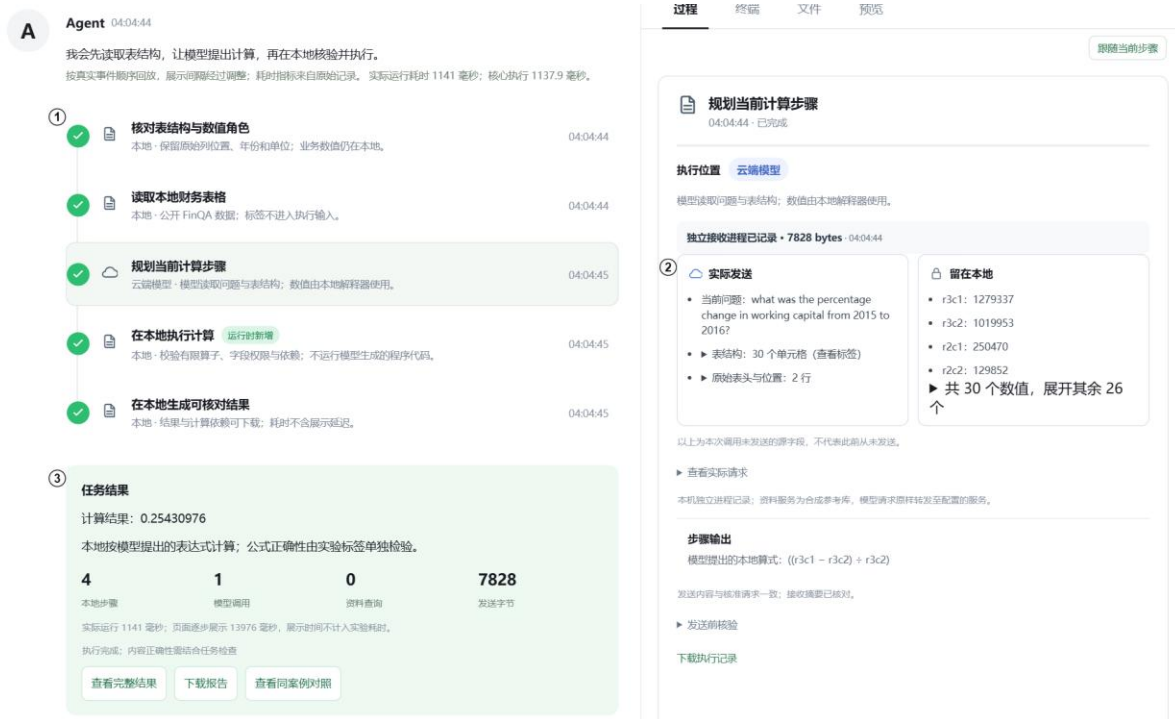


图 2 公开表格开发示例的真实执行记录。① 从原表核对结构与数值角色；② 右栏显示模型实际收到的问题与表结构，以及本次保留的业务数值；③ 本地执行算式并返回结果。截图截去导航与源文件卡片，并添加编号；此示例不计入测试。

3 交互演示

演示采用独立研究界面，沿用客户端任务布局。中栏自动推进，右栏显示选中步骤的执行位置、输入、保留字段和接收记录。观众可以取消跟随查看某一步，也可下载请求体、计算表达式和完整轨迹。新增步骤由实际工具调用触发，不由预设动画生成。

首先运行本地规则适用的标准条款，修改金额后重新执行，观察数值变化及零外部调用。随后运行补充事实案例：每日 230 元的条款最初缺少延期天数，模型申请后获得 5 天，再完成计算。选择前后两次分析，可比较接收内容的变化；案例也说明数值正确不保证解释正确。

第三个场景在运行中查找指定版本资料：查询仅含编号和版本，后续分析增加返回正文；观众可在发送前撤销。第四个场景使用公开年报表格，先查看年份表头与业务数值的分离，再比较全量、按需和本地计算。营运资金示例读取 2015 年与 2016 年的两个本地数值，得到约 25.43% 的增幅；云端看到列的含义，但不接收数值或计算结果。

实现由 Node.js 控制器、独立 HTTP 接收进程和双语网页构成。合成 PDF 用 PDF.js 读取并与登记字段核对；公开表格读取结构化文件，PDF 仅作预览。无密钥时可执行有限本地规则或重放保存的真实调用；现场 DeepSeek 运行使用读者自己的密钥。展示间隔单独放慢，实验耗时不含这部分时间。

4 实验设计

实验前冻结源码、输入、标签与配置。60 页来自 57 份公司年度报告、38 家公司；每例五种方法、重复两次，共 600 次任务。采用 deepseek-flash、temperature=0、每次最多 900 个输出 token，四任务并发；方法次序轮换。1,210 次真实调用均与请求及响应摘要对应。

表格取自 FinQA [6] 官方数据的受限子集：问题只依赖表格，标注算子受支持，操作数可唯一定位。按固定摘要顺序选择页面，并排除此前测试与开发使用的全部公司年度报告。旧批次的六个开发题及六个诊断题只用于修复接口与表头；本轮测试没有按模型表现筛题。数据可能在模型训练中出现。

五种方法共享修正后的表结构、权限与解释器。全量与本地计算各规划一次，前者传输数值，后者只传结构；按需方法先申请单元格。无数值复核与分歧补充均先独立提出三个方案，再在不一致时复核；两者的复核提示相同，区别是否提供式 (4) 的字段。默认采用单次本地计算。

按原始 exe_ans 评分，绝对容差为 0.00005；错误及异常均计入分母。业务数值按实际请求累计，重复发送重复计数；标签、年份和单位仍可见。耗时包含结构重建、调用与检查，排除界面放慢。两次结果先在案例内平均，再按年度报告做 4,000 次配对聚类重采样。

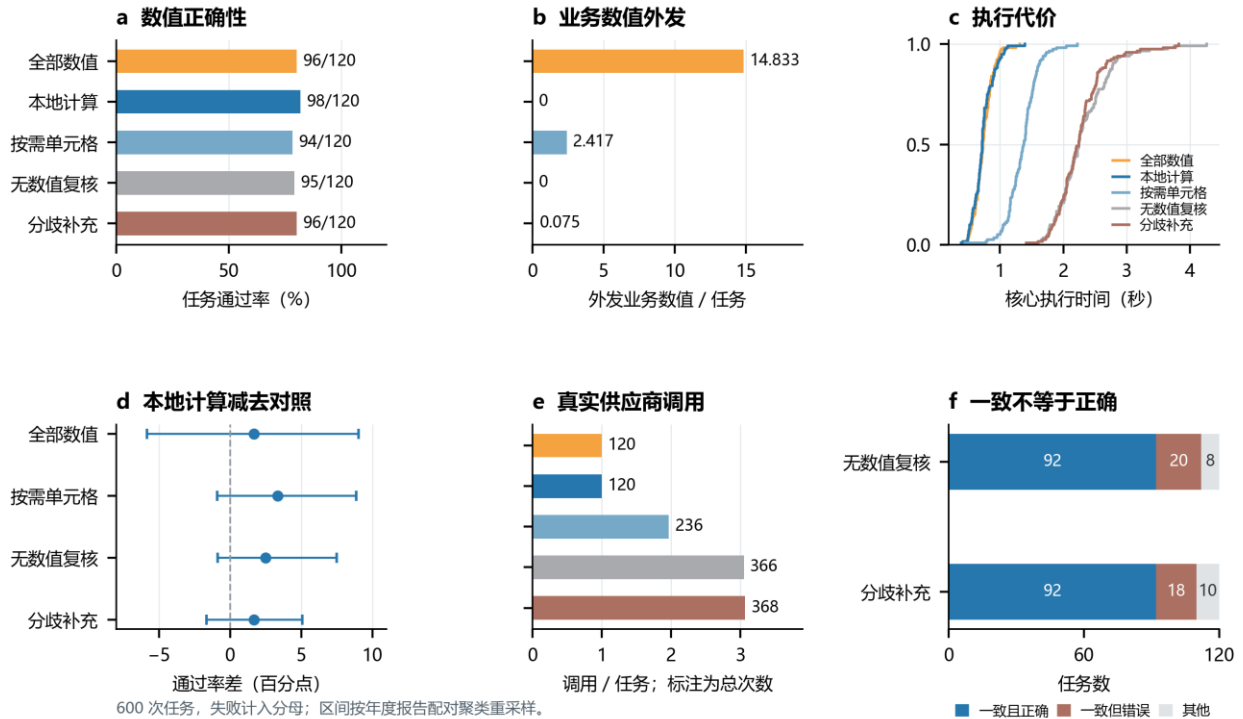


图3 最新冻结表格实验。a-c，正确性、业务数值外发与执行耗时；d，本地计算相对各对照的通过率差及报告聚类区间；e，实际调用；f，结构一致但答案错误的任务。每种方法120次任务，失败计入分母；区间按年度报告配对聚类重采样。

5 结果与适用边界

图3显示单次本地计算通过98/120次，全部数值为96/120，按需单元格为94/120。对应每任务业务数值发送量为0、14.833和2.417；调用总数为120、120和236。本地计算相对全量的通过率差为1.67个百分点，描述性95%区间为[-5.83, 9.02]，不能据此声称更优或不劣。

分歧补充通过96/120次，无数值复核为95/120；前者发送0.075个业务数值/任务，两者分别调用368和366次。与单次本地计算相比，多方案方法未呈现准确率收益，核心耗时中位数约由0.72秒增至2.25秒。分歧补充中有18次任务的三个方案结构一致却答案错误；一致性只能决定是否复核，不能作为正确性证书。

历史合成强对照另保留256次任务。按需补充、关闭补充、获准业务字段全量、数值预取分别通过57/64、50/64、58/64、58/64次；多余字段均值分别为0.031、0、0.563、0.063。按需补充需80次调用，后两种预取各为64次。当必要数值容易预先识别时，预取仍是有竞争力的基线。该集合已用于开发，不与新表格结果合并。

旧表格转换曾把部分年份表头当作业务数值隐藏，破坏了无数值视图的列含义。本轮修复输入构造并使用新报告重新测试；旧协议、原始结果和更正说明全部保留，不能将跨批次差异归因于新算法。边界测试另外覆盖撤权、源值改写、接收方替换、凭据重放和程序越权。

6 开放实现与局限

仓库公开独立界面、执行器、冻结协议、原始调用及失败样本；600份最新记录可无密钥复算。读者可重放真实调用，或配置自己的DeepSeek密钥重新执行。商业客户端与凭据不在开源包内。自由文本盲审材料已准备，但真人评阅尚未完成，本文只报告结构化与数值结果。

实验只涉及一个模型和受限公开子集，不代表真实部署效果。我们未完成相关系统的端到端复现比较，也未证明语义最小性或任意工具的安全性。本文的演示价值在于把执行决策与实际数据流联系起来，让共享收益、额外调用和失败原因都能被检查。

代码与复现：<https://github.com/Zane-0260907/stepwise-disclosure-demo>

参考文献

- [1] G. Yu 等. PlanTwin: Privacy-Preserving Planning Abstractions for Cloud-Assisted LLM Agents. arXiv:2603.18377, 2026.
- [2] W. Li, Y. Xu. ToolMinimize: Auditing and Rewriting LLM Agent Tool Calls to Minimize Privacy Exposure. arXiv:2608.24957, 2026.
- [3] H. Yu 等. MINIM: Privacy-Aware Minimal View for Agents via Trusted Local Sanitization. arXiv:2606.13949, 2026.
- [4] L. Gao 等. PAL: Program-aided Language Models. ICML, 2023, 10764–10799.
- [5] X. Wang 等. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR, 2023.
- [6] Z. Chen 等. FinQA: A Dataset of Numerical Reasoning over Financial Data. EMNLP, 2021, 3697–3711.